

Highlights

Rethinking Runtime–Energy Trade-offs through the Interplay of Parallelism and Power Limiting

- Parallelism and power limits form tightly coupled control variables
- Energy efficiency is governed by power distribution and scaling behaviour
- Independent tuning leads to suboptimal runtime–energy trade-offs
- The trade-off landscape is irregular, non-convex, and platform-dependent
- Multi-objective optimisation is essential for identifying runtime–energy trade-offs

Rethinking Runtime–Energy Trade-offs through the Interplay of Parallelism and Power Limiting

ARTICLE INFO

Keywords:

Parallelism
Power limiting
Frequency scaling
Energy consumption
Energy efficiency
Green software
Sustainability

ABSTRACT

Understanding and improving the energy efficiency of software requires insight into how software execution interacts with increasingly complex hardware power management mechanisms. Although energy is ultimately consumed by hardware, software determines *how* that hardware is exercised; not only through implementation details, but also by execution parameters such as parallelism and power limits. These controls are typically studied in isolation, under the assumption that their effects compose. However, modern processors dynamically distribute a shared power budget across components, giving rise to non-obvious interactions.

Using a measurement methodology that isolates program-induced energy from baseline power draw, we study how parallelism and power limits jointly influence runtime performance and energy consumption across multiple workload classes and hardware platforms. We show that the combined effects of parallelism and power limiting cannot be inferred from their individual trends: coordinated tuning reveals runtime–energy trade-offs that remain hidden when each control is adjusted in isolation.

Our results demonstrate that optimising for runtime performance or energy consumption in isolation can lead to suboptimal configurations. Instead, both objectives must be considered jointly: configurations that diverge slightly from the runtime optimum can yield substantial energy savings, and vice versa. This shows that efficient execution is not characterised by a single optimum, but by a set of trade-offs between competing objectives. Overall, our findings highlight the need for joint tuning of parallelism and power limits, and establish multi-objective optimisation as a necessary approach for identifying configurations that balance runtime performance and energy efficiency.

1. Introduction

Software does not consume energy; the hardware executing it does. However, software determines how that hardware is exercised, not only through implementation choices, but also through execution parameters such as parallelism and power limits. As hardware platforms have become increasingly complex, it has become more difficult to predict how these software-level decisions translate into energy use. This suggests that improving software energy efficiency requires not only code-level optimisation, but also control over *how* software is executed on the hardware.

A large body of prior work studies execution-level controls such as parallelism [1–3], frequency [4–10], and power limiting [11–14]. When considered in isolation, these controls often exhibit predictable behaviour: increasing parallelism tends to improve both runtime performance and energy efficiency, whereas reducing frequency or tightening power limits often reduces energy consumption at the cost of longer execution times. This creates the impression that these controls can be tuned independently, and composed sequentially, to obtain energy-efficient configurations.

Modern processors, however, operate under a shared power budget that is dynamically distributed across cores, caches, interconnects, and memory controllers. Activating additional cores increases power demand and can trigger frequency scaling, while tightening the power limit constrains all active components simultaneously. As a result, adjusting either control reshapes how power is allocated across the processor, and how subcomponents contribute to performance and energy use. These mechanisms suggest that

the effects of parallelism and power limiting are not compositional, and that independent tuning may fail to identify efficient configurations.

In this paper, we investigate how parallelism and power limits jointly shape runtime and energy across multiple hardware platforms. Using a measurement methodology that isolates program-induced energy from baseline power draw, we analyse three representative workload classes that exercise the hardware in different ways. Crucially, we demonstrate that the combined effects of parallelism and power limiting cannot be predicted from their individual trends: coordinated tuning reveals runtime–energy trade-offs that remain hidden when these controls are considered independently.

A key implication is that optimising for runtime performance or energy efficiency in isolation is insufficient. Across workloads, we observe extended configuration spaces in which runtime remains nearly constant while energy varies substantially, and vice versa. Near-optimal runtime performance can often be achieved at significantly lower energy cost, while a modest increase in energy consumption can yield disproportionate performance gains. Efficient execution is therefore not characterised by a single optimum, but by a set of trade-offs between competing objectives.

This motivates a multi-objective perspective that considers runtime and energy jointly. Rather than identifying a single optimal configuration, we characterise a trade-off surface between both objectives. To capture these trade-offs, we use Z-plots to represent runtime performance and energy consumption jointly across the configuration space. This exposes behaviour that is not visible when adjusting either control, or either objective, in isolation.

Our contributions are:

- An energy measurement methodology that separates program-induced energy consumption from baseline power draw, enabling meaningful comparison across configurations and hardware platforms.
- An experimental analysis of both the individual and joint effects of parallelism and power limiting on runtime and energy consumption across multiple workload classes and hardware platforms.
- Evidence that the effects of these controls are non-compositional, and that simultaneous tuning is required to identify configurations that balance runtime performance and energy efficiency.
- A multi-objective analysis of runtime–energy trade-offs, supported by Z-plots, that characterise runtime–energy trade-offs across the configuration space.
- We identify the role of hardware power management, in particular core and uncore frequency scaling, as a key factor governing these trade-offs.

2. Defining “energy consumption”

Understanding how hardware consumes energy is essential not only for explaining how parallelism and power limits affect energy use, but also for understanding how software activity drives hardware energy consumption. This knowledge establishes how software energy consumption should be measured and attributed, and provides the basis for interpreting the resulting software–hardware interactions.

Hardware primarily consumes energy through power dissipation, which occurs when electrical energy is converted into heat. Commonly, a distinction is made between dynamic power and static power [15]. Dynamic power is program-induced and largely caused by transistor switching, whereas static power corresponds to the baseline power required to keep the hardware, operating system, and necessary background tasks operational.

Measured energy consumption is thus a combination of program-induced energy and idle energy use:

$$E_{\text{measured}} = E_{\text{program}} + P_{\text{idle}} \cdot t_{\text{program}}$$

Where E is energy, P is power, and t is time. Because idle power draw is relatively constant, measured energy scales linearly with runtime. This implicitly favours faster programs, even if they use hardware inefficiently.

Still, many energy evaluations include idle power draw, often implicitly or unknowingly. This captures energy that is unrelated to the program itself and can distort comparisons between configurations [16, 17]. Some approaches compensate by padding runtimes to equal length [18], but this introduces further artefacts caused by background activities that can trigger after program completion, such as memory reclamation and storage wear levelling.

This discussion reflects two perspectives on energy attribution from the field of life-cycle assessment: attributional and consequential analysis [19, 20]. An attributional perspective accounts for all resource consumption, including that which is outside the influence of the system under test. A consequential perspective measures only the changes caused by an intervention. Software energy evaluation is inherently consequential: we are interested in how energy consumption changes when a program or its execution is modified [21, 22]. Idle power is external to program behaviour and should therefore not be attributed to it.

This suggests that idle energy consumption should be excluded from energy measurements, so that results reflect changes actually caused by the program and its configuration. This correction removes the runtime dependency and eliminates the need to artificially pad executions.

3. Energy-related hardware behaviour

Modern processors consist of a collection of semi-independent domains—cores, caches, interconnects, the memory controller—that share a single power budget. The power budget represents the maximum electrical power the processor package is allowed to draw at any moment, typically bounded by thermal design limits and enforced by software limits or on-chip power management hardware. How power is distributed among domains depends on utilisation, temperature, and internal power governor heuristics. Understanding how these domains interact, and how power is distributed among them, is essential for interpreting the energy-related behaviour observed in this study.

Processors continuously adjust operating frequencies based on utilisation, thermal headroom, and power availability. Activating more cores typically lowers per-core frequency. Applying a power limit constrains all components simultaneously, often reducing frequency across both active and idle domains. Because power draw scales roughly cubically with frequency [23], small frequency changes can have a disproportionate effect on energy consumption.

A key distinction is between core and uncore components. Cores contain compute units and private caches, while the uncore includes last-level caches, interconnects, and the memory controller. On some architectures, core and uncore frequencies scale together; on others, they are adjusted independently. Thus, performance critically depends on effective frequency scaling across domains: compute-heavy workloads depend primarily on core frequency, whereas memory-bound workloads hinge on uncore performance.

These mechanisms explain why identical control settings can produce different runtime–energy trade-offs across workload classes and hardware platforms.

4. Experimental setup

To investigate these hardware behaviours, we evaluate two distinct hardware platforms: a high-power desktop and a low-power Odroid-H4 single-board computer. Table 1 summarises key hardware characteristics. The benchmark scripts

	Desktop	Odroid-H4
Processor	Core Ultra 7 265K	Processor N97
Core configuration	8 P-cores, 12 E-cores	4 cores (4 threads)
Base-boost frequency	3.9–5.4 GHz	2.0–3.6 GHz
L1 cache (per core)	192 kB	96 kB
Sustained power limit	125 W	12 W
System memory	32 GB DDR5-6400	16 GB DDR5-5600
Theoretical bandwidth	51.20 GB/s	44.80 GB/s

Table 1

Key hardware characteristics of the desktop system and Odroid-H4 single-board computer. Processor frequencies on the desktop system are only given for performance cores, as we do not use the efficiency cores in our benchmarks.

and results are available in the artefact [24]. The artefact also includes results from a laptop system, but we omit it from our analysis because its behaviour closely mirrors the desktop system and provides no additional insights.

4.1. Workload classes

On each platform we evaluate three synthetic workload classes, each representing a different hardware bottleneck.

- **CPU-saturating:** a compute-heavy N-body simulation dominated by arithmetic throughput. (Section 5)
- **CPU-underutilising:** a disk write benchmark limited by throughput of external storage. (Section 6)
- **Interleaved:** a memory copy benchmark whose performance depends on the interaction between the processor and system memory. (Section 7)

Real applications typically exhibit a mixture of these phases, and the dominant bottleneck depends on program inputs, system configuration, and hardware characteristics. Consequently, the same application may be limited by different resources on different systems. By isolating a dominant bottleneck with each synthetic benchmark, we can systematically evaluate how system configuration and hardware power management influence runtime and energy behaviour.

4.2. Energy measurement instrumentation

We measure CPU energy consumption using the Running Average Power Limit (RAPL) interface. Although initially based on estimation models, RAPL now provides running counters that are updated every 1 ms for accurate energy measurements of the package and subdomains [25]. RAPL has been available on Intel architectures since their Sandy Bridge architecture (2011). AMD supports a similar interface since their Zen architecture (2017).

In practice, it is impossible to perfectly separate idle power draw from program-induced power draw. We approximate idle power by measuring RAPL energy consumption at one second intervals over a three-minute period while the system is idle; taking the median as the baseline. Because idle power itself can depend on the power limit, this procedure is repeated for each power-cap configuration.

Program-induced energy is computed as:

$$E_{\text{program}} = E_{\text{measured}} - P_{\text{idle}} \cdot t_{\text{program}}$$

This correction ensures that energy comparisons reflect changes attributable to the program and its configuration, rather than differences in idle energy arising from variations in program runtime.

Although whole-system energy measurements would be ideal for capturing all component-level interactions, its practicality is limited. Measuring at the socket is difficult due to low sampling frequency and power supply efficiency curves. Direct measurement of internal wiring offers better accuracy, but requires special equipment and physical access.

For validation, we perform whole-system measurements on the Odroid-H4 using an INA260 I²C power monitor integrated into the internal wiring [26]. Although sampling frequency is limited to 80 Hz, we empirically observe that whole-system energy trends closely match CPU energy trends across all three benchmarks, confirming that RAPL provides sufficient insight for the purposes of our study.

5. CPU-saturating workload

To evaluate a CPU-saturating workload we choose the naive N-body algorithm, whose performance is dominated by arithmetic throughput. The algorithm parallelises near-perfectly and its working set is small enough to fit entirely within each core's L1 cache, minimising interference from shared resources. The algorithm is implemented in *Single assignment C* (SaC) [27–29], which generates highly optimised sequential and parallel code from a single source.

To account for performance and cache-size differences between the two platforms, we use a problem size of 4096 bodies for the desktop system and 1024 bodies for the Odroid-H4. Because these working sets are relatively small, a single iteration of the N-body kernel completes too quickly for accurate energy measurement. Therefore, each benchmark run repeats the kernel 100 times, ensuring that runtimes are sufficiently long to obtain accurate energy measurements. After five warm-up runs we record runtime, energy consumption, and relevant performance counters across fifty benchmark runs. We also track CPU temperatures, which remain below thermal limits for all benchmarks.

We first investigate parallelism and power limiting in isolation, to understand their individual effects on hardware-related energy behaviour. On the basis of these insights we then investigate their joint effect on runtime and energy.

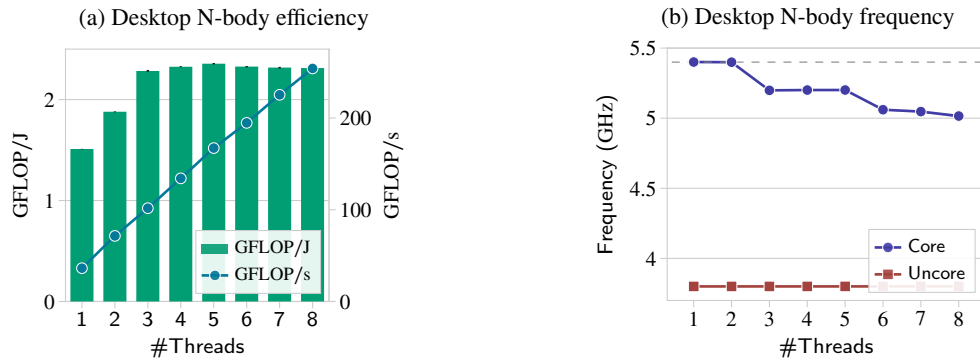


Figure 1: Desktop N-body workload median runtime performance and energy efficiency (a) and processor frequency (b) when adjusting the thread count, using the default power limit of 120 W. Error bars report the interquartile range.

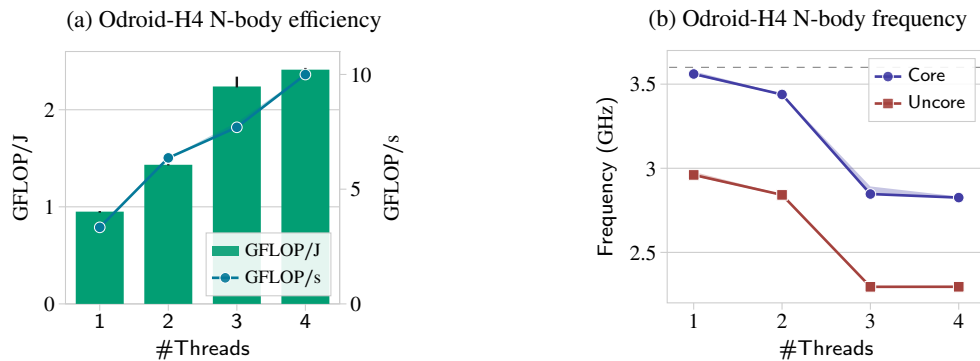


Figure 2: Odroid-H4 N-body workload median runtime performance and energy efficiency (a) and processor frequency (b) when adjusting the thread count, using the default power limit of 12 W. Error bars report the interquartile range.

5.1. Parallelism is a free lunch

Because the N-body workload is core-local and parallelises nearly perfectly, runtime performance is expected to scale nearly linearly with thread count. Indeed, Figure 1a confirms that floating-point throughput on the desktop system increases proportionally with the number of threads. Although dividing the work over threads decreases runtime, the total number of operations needed to compute the result stays the same. Thus, parallelism alone does not inherently decrease total energy use.

Yet, Figure 1a shows that energy efficiency, in floating-point operations per Joule, also improves with thread count. The increase in energy efficiency is mainly driven by a decrease in operating frequency, which is necessary for the CPU to remain within its power envelope and thermal limits as more cores become active. Figure 1b confirms this: the sequential configuration operates at approximately 5.4 GHz, dropping to 5 GHz when all eight cores are active. Because power draw scales roughly cubically with frequency [23], this modest frequency reduction has a significant positive impact on energy efficiency.

The Odroid-H4 exhibits a similar trend (Figure 2a). Because the Odroid-H4 is designed for mobile applications, its frequency is downscaled more aggressively compared to the high-powered desktop system. Figure 2b shows that, as

thread count increases, frequency drops from the processor's maximum boost frequency of 3.6 GHz down to 2.6 GHz. This significant frequency reduction results in steeper energy efficiency gains compared to the desktop system, at the cost of less-than-linear runtime scaling. Additionally, since the benchmark does not stress the uncore, its downscaling produces further energy savings without impacting runtime performance.

Parallelism improves not only runtime performance, but also energy efficiency, making it effectively a “free lunch” for this specific class of compute-intensive workloads. Mobile-class devices such as the Odroid-H4 show an even steeper gain as a consequence of more aggressive frequency scaling behaviour, which reveals how architectural differences influence energy-related behaviour.

5.2. Power limiting risks unpredictable behaviour

Power limits constrain the entire CPU package, triggering frequency and voltage scaling as the configured threshold is approached. In an ideal scenario, the power governor would first reduce the performance of underutilised components, thereby minimising the impact on performance-critical components for as long as possible.

In practice, however, the desktop system does not exhibit this ideal behaviour. Figure 3b shows that core and uncore frequency decrease at a similar rate as the power limit

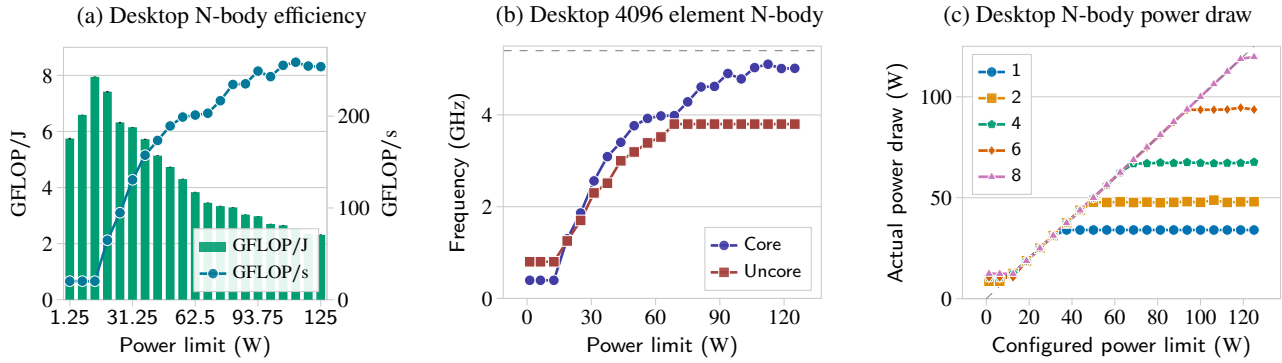


Figure 3: Desktop N-body workload median runtime performance and energy efficiency (a), processor frequency (b), and power draw (c) when adjusting the power limit, using all eight available performance cores. Error bars report the interquartile range.

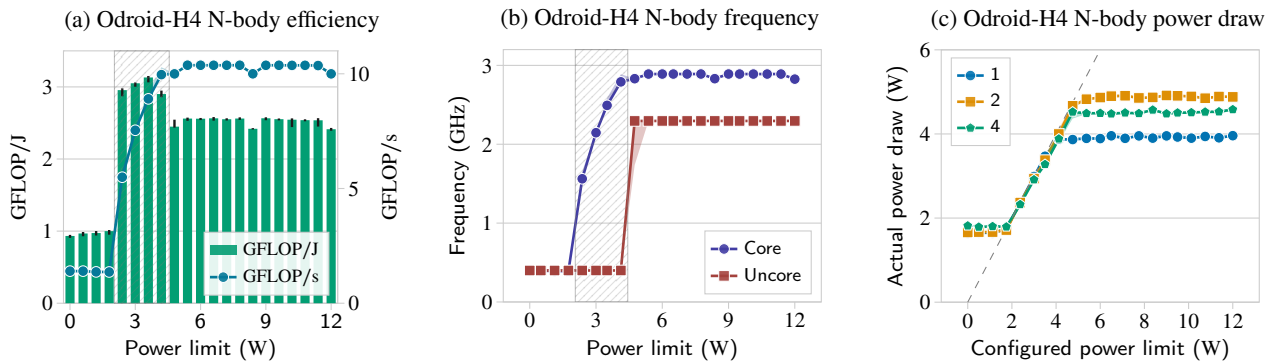


Figure 4: Odroid-H4 N-body workload median runtime performance and energy efficiency (a), processor frequency (b), and power draw (c) when adjusting the power limit, using all four available cores. Error bars report the interquartile range.

tightens. Runtime performance degrades in line with the observed frequency reduction, confirming that the decrease in runtime performance is directly attributable to the decrease in core frequency. As expected, energy efficiency is significantly improved by the decrease in frequency.

Because the N-body workload does not stress uncore components, in principle energy efficiency could have been improved without affecting runtime by only lowering uncore frequency. This indicates that power is not redirected to favour performance-critical components, leaving potential runtime performance unrealised. The underlying cause of this suboptimal trade-off remains unclear, and may arise from hardware constraints, system configuration, or power governor policies. Regardless of its origin, this observation highlights that effective power management must be sufficiently selective to avoid sacrificing attainable performance.

Energy efficiency follows an expected upward trend as the power limit decreases, until the power limit becomes extremely tight. A key observation in Figure 3c is that below 12.5 W, the CPU can no longer sustain all eight active cores alongside all its other components. Although the configured power limit nominally specifies an upper bound, the processor fails to reach it in practice: actual power draw remains capped at approximately 12.5 W regardless of further tightening of the limit. This indicates severe resource starvation,

necessitating the processor to alternate power availability among components to remain operational, resulting in considerable scheduler-level overhead. The resulting instability leads to a steep decline in both runtime performance and energy efficiency. These effects establish a practical lower bound on usable power limit configurations: once the limit becomes too restrictive, performance degradation outweighs potential energy savings.

Overall, power limiting highlights that optimising for runtime performance and energy efficiency can be directly opposed. Maximising performance of only one metric does not necessarily result in an ideal trade-off, because at either extreme a small runtime sacrifice can yield disproportionate energy efficiency gains, and vice versa. For example, although a power limit of 12.5 W minimises energy, increasing the power limit to 31.25 W more than quadruples runtime performance, at the cost of a relatively minor energy overhead. In practice, depending on the relative importance of runtime performance and energy efficiency, a conscious choice has to be made about how much runtime performance should be traded for energy efficiency gains.

5.3. Hardware selection shapes frequency scaling

On the desktop's CPU, core and uncore frequencies scale at a similar rate across power-cap settings, allowing clean

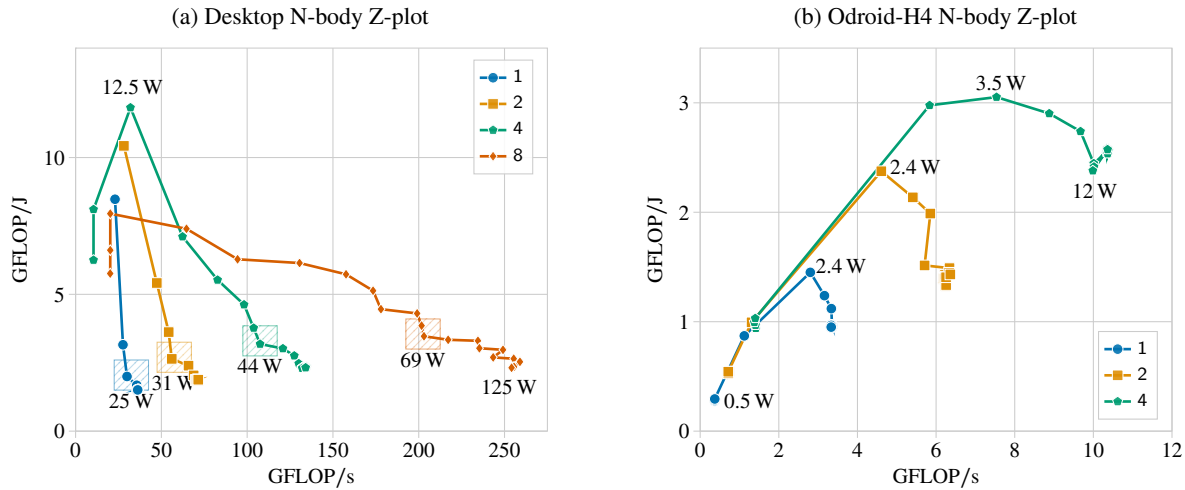


Figure 5: Z-plots combining parallelism and power limiting configurations for the N-body workload, on the desktop system (a) and the Odroid-H4 (b). Lines connect configurations with equal thread count, while movement along a curve from left to right reflects the effect of increasing the power limit. Exact power limits are given for relevant configurations.

attribution of performance changes to frequency scaling. In contrast, the Odroid-H4 exhibits a more decoupled core and uncore frequency management scheme. When combined with power limiting, this leads to complex behaviour, as core and uncore frequencies are affected non-linearly.

We observe in Figure 4b that uncore frequency on the Odroid-H4 starts to decrease at a power limit of 4.8 W, while core frequency actually slightly increases. Uncore frequency reaches its minimum observed value at 3.6 W, whereas core frequency remains relatively high until reaching its minimum observed value at 1.8 W. Since the N-body program relies only on core-local compute performance, the decrease in uncore frequency yields energy efficiency gains without a runtime cost. The three most energy-efficient configurations in Figure 4a correspond exactly to the three cases between 2.4 W and 3.6 W where uncore frequency is reduced while core frequency remains relatively high.

Additionally, we observe that performance remains unchanged for power limit configurations between 4 W and 12 W. Figure 4c shows that actual power draw never exceeds 4 W, even when the configured limit is increased to its 12 W maximum. This could suggest that firmware-level heuristics of this low-power device favour conservative power delivery, or that unused subsystems remain properly inactive.

These results highlight that effective power governing can yield energy efficiency gains without sacrificing runtime performance, by selectively reducing the frequency of under-utilised components. Effective frequency scaling heuristics can be a powerful tool, but they also introduce complexity and unpredictability in how power limits affect performance and energy use. Critically, architectural details such as core–uncore frequency coupling play an important role in shaping the energy profile of an application.

5.4. Energy efficiency requires a trade-off

So far, we have examined the effects of parallelism and power limiting in isolation. While these results already show hardware-dependent and sometimes counterintuitive behaviour, they do not reveal how runtime performance and energy efficiency evolve when both controls are adjusted simultaneously. This raises two key questions: do parallelism and power limits act independently, and can configurations that are optimal for each control in isolation be combined to obtain a globally optimal configuration? Answering these questions requires analysing the joint configuration space defined by parallelism and power limits, with runtime performance and energy efficiency as competing objectives.

A natural way to represent this configuration space is through three-dimensional surfaces. However, such representations are difficult to interpret in this context. First, they require separate plots for each objective, obscuring the trade-offs between runtime performance and energy efficiency. Second, identifying trade-offs becomes visually challenging when the landscape is non-monotonic and contains multiple local optima. Consequently, such visualisations make it difficult to reason about which configurations provide the best balance between runtime performance and energy efficiency.

To make these trade-offs explicit, Figure 5 presents Z-plots [30], which combine runtime performance (x-axis) and energy efficiency (y-axis) across the entire configuration space. Lines connect configurations with equal thread count, while movement along a curve reflects the effect of varying the power limit. Moving to the right indicates higher runtime performance, while moving upwards indicates improved energy efficiency. An ideal configuration would thus lie in the upper-right corner, maximising both objectives.

The plots reveal that such an ideal configuration does not generally exist. On the desktop system, the structure of this trade-off is largely monotonic: for a fixed thread count, tightening the power limit consistently improves energy

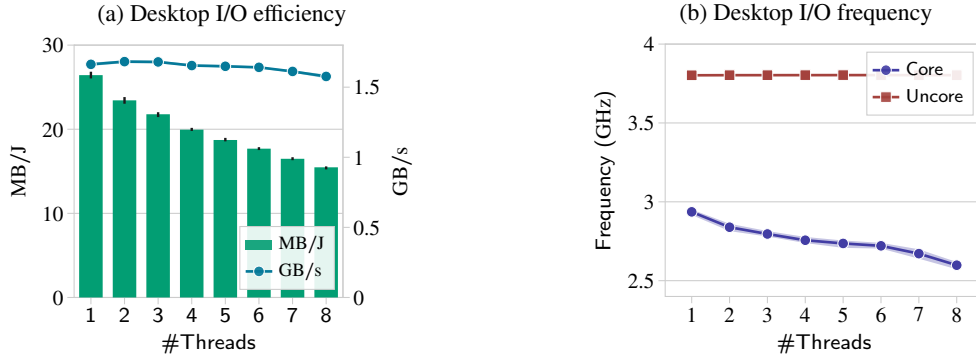


Figure 6: Desktop I/O-bound workload median runtime performance and energy efficiency (a) and processor frequency (b) when adjusting the thread count, using the default power limit of 120 W. Error bars report the interquartile range.

efficiency at the cost of runtime performance. For lower thread counts the runtime penalty is smaller, as fewer active cores reduce the overall power demand.

If the two controls are truly independent, one would expect the most energy-efficient configuration to combine the highest thread count with the lowest feasible power limit (based on the individual analyses in previous sections). The Z-plots reveal that this assumption does not hold. Instead, the globally most energy-efficient configuration is obtained using four threads, combined with a 12.5 W power limit. Namely, the optimum emerges from the interaction between both controls: parallelism decreases frequency—thereby increasing energy efficiency—but it also increases power demand, which limits how much the power limit can be tightened. Therefore, the global optimum cannot be inferred from the individual effects of each control.

Moreover, the trade-off is not smooth. Across all thread counts we observe locally suboptimal regions, where adjusting the power limit can improve runtime performance or energy efficiency for a limited trade-off. These regions (highlighted in Figure 5a) are identified by a “kink” in the curve. As the number of active cores decreases, this region shifts towards lower power limits, reflecting the reduced power required to sustain active cores.

On the Odroid-H4 (Figure 5b), the interaction looks very different. Across thread counts, moderate reductions in the power limit initially improve energy efficiency with limited runtime impact, offering attractive configurations when both objectives matter. Beyond a certain point, however, further reductions lead to a simultaneous degradation in runtime performance and energy efficiency. As on the desktop system, the location of this transition depends on thread count, occurring at higher power limits when more cores are active.

Taken together, these results show that power limiting determines the shape of the runtime–energy trade-off, while parallelism determines how effectively the available power budget is utilised. Across configurations, increasing parallelism continues to improve runtime performance and energy efficiency, so long as the power limit remains sufficiently high to sustain active cores. Under tight power constraints, however, sustaining many active cores becomes

inefficient, and reducing parallelism can improve both objectives. Overall, Z-plots reveal that energy-efficient execution for this workload cannot be achieved by tuning either control in isolation, nor by composing independently optimal configurations, but requires jointly balancing parallelism and power limits to navigate the runtime–energy trade-off.

6. CPU-underutilising workload

At the opposite end of the spectrum from compute-bound workloads are I/O-bound programs, whose performance is primarily limited by external storage devices rather than by compute performance. On both platforms we write an array of 10 million doubles to disk, which is large enough to exceed the CPU caches and trigger sustained I/O activity. This workload allows us to test whether parallelism and power limiting still influence energy consumption when the processor is largely idle. In principle, their runtime should be insensitive to computational performance, as long as the CPU can keep up with I/O demand. This suggests the possibility of reducing computational performance while preserving throughput, lowering energy consumption with minimal impact on runtime.

6.1. Parallelism hurts I/O throughput

Because the workload is not bound by compute performance, increasing the number of threads is not expected to improve runtime performance. However, activating additional cores increases power demand, even if those cores are mostly idle. Indeed, Figure 6a shows that throughput remains roughly constant across thread counts, slightly decreasing when additional cores are activated. This decline is not explained by frequency scaling alone: Figure 6b shows only a 250 MHz drop. Instead, the dominant cause is software-level resource contention. Threads compete for shared I/O resources, introducing synchronisation overheads that increase latency and extend runtime.

While the runtime penalty is modest, energy efficiency degrades substantially as thread count increases. This effect is driven primarily by the increased power demand of keeping more cores active, and by frequent context switching,

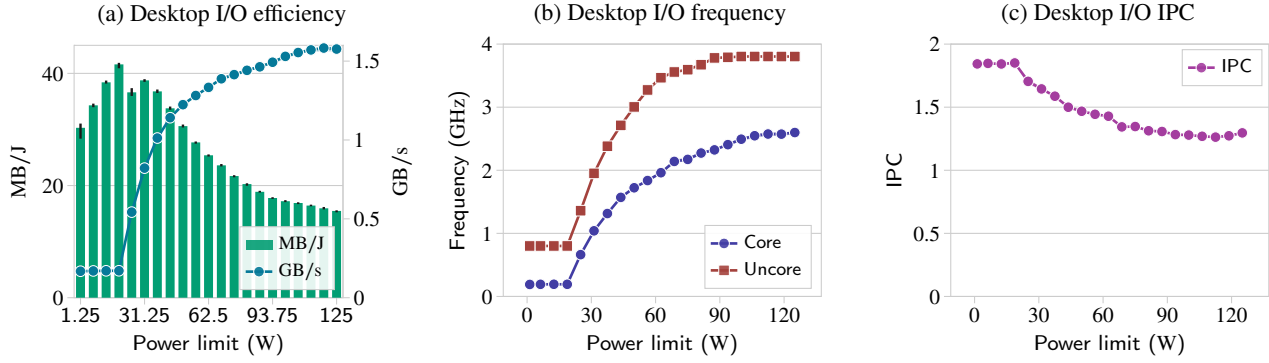


Figure 7: Desktop I/O-bound workload median runtime performance and energy efficiency (a), processor frequency (b), and IPC (c) when adjusting the power limit, using all eight available performance cores. Error bars report the interquartile range.

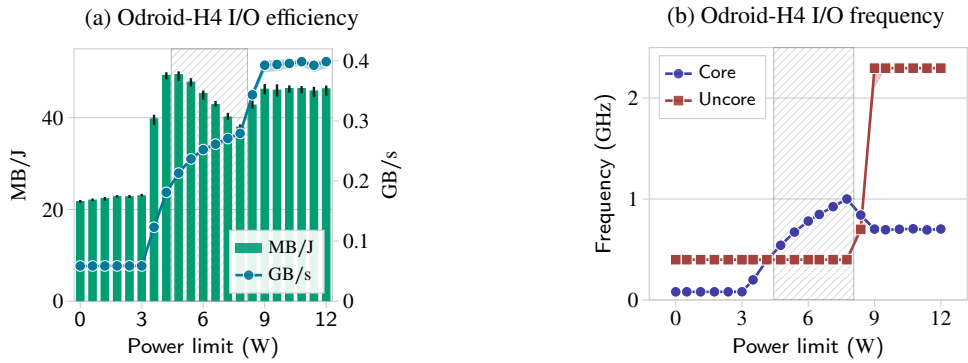


Figure 8: Odroid-H4 I/O-bound workload median runtime performance and energy efficiency (a) and processor frequency (b) when adjusting the power limit, using all four available cores. Error bars report the interquartile range.

which prevents cores from entering low-power states. These results show that, even when cores are mostly idle, activating more cores can significantly increase energy consumption without improving throughput.

Although the sequential configuration maximises both objectives, we use the configurations utilising all cores for subsequent power-limit experiments, to ensure that the entire power budget can be utilised.

6.2. Is power limiting a free lunch?

If I/O throughput is truly independent of CPU speed, lowering the power limit should reduce energy consumption without affecting runtime. In practice, however, the processor remains responsible for data preparation and operating-system interactions, and thus cannot be slowed arbitrarily.

Figure 7a shows that runtime performance decreases as the power limit is reduced, contradicting the naive expectation. Once CPU performance drops below the threshold needed to queue and service I/O requests efficiently, throughput declines. Energy efficiency initially improves as wasted cycles are eliminated, but deteriorates once the power limit becomes too restrictive. Figure 7c confirms this: as the power limit is reduced to 31.25 W, instructions per cycle (IPC) improves at a similar rate as energy efficiency, indicating that fewer cycles are wasted waiting for I/O.

This case demonstrates that, even though the processor is not fully saturated, it must retain sufficient performance to sustain data movement and kernel activity. As a result, the energy-minimising power limit for this workload lies higher compared to the compute-bound workload, which is the most energy-efficient at a power limit of 12.5 W (Figure 3a).

6.3. Frequencies reshape power distribution

On the Odroid-H4, power limiting again exhibits non-linear behaviour due to the decoupling of core and uncore frequencies. Figure 8a shows a dip in energy efficiency at intermediate power limits, which coincides with configurations where core frequency exceeds uncore frequency in Figure 8b. Rather than uniformly reducing operating frequencies, tightening the power limit redistributes the available power budget across processor domains. In this case, the additional power headroom from decreased uncore frequency enables an increase in core frequency, counteracting the intended energy savings.

These results illustrate that power limits influence not only the total power budget, but also how power is distributed across components. Power limiting acts as a higher-level abstraction that allows power governors to adapt operating frequencies across domains, without explicit frequency management. This is particularly valuable given increasing

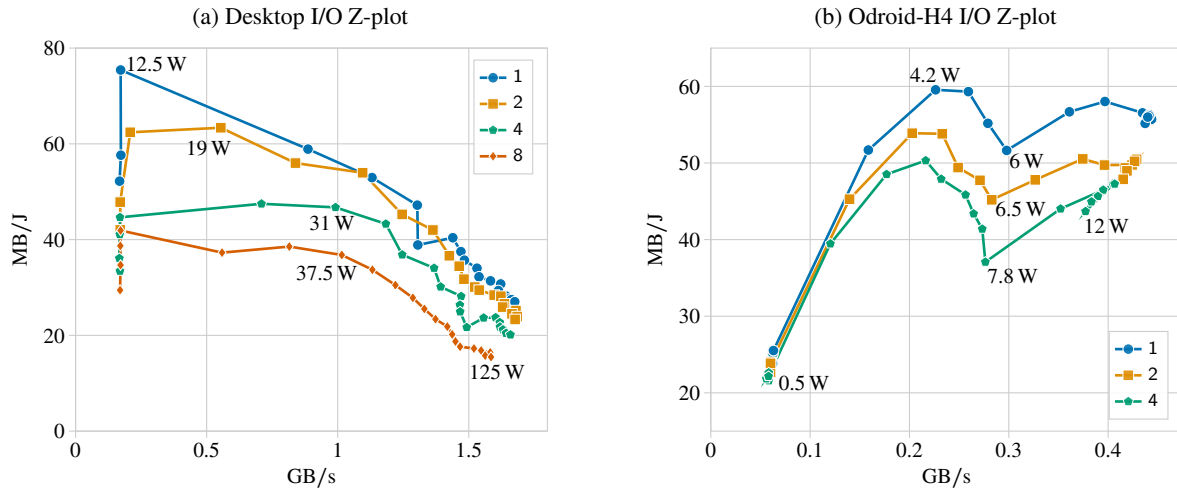


Figure 9: Z-plots combining parallelism and power limiting configurations for the I/O-bound workload, on the desktop system (a) and the Odroid-H4 (b). Lines connect configurations with equal thread count, while movement along a curve from left to right reflects the effect of increasing the power limit. Exact power limits are given for relevant configurations.

hardware complexity and heterogeneous platforms, where manual frequency control is both time-consuming and difficult to generalise across systems.

For I/O-bound workloads, where throughput depends primarily on storage and uncore activity, increasing core frequency provides little benefit. Indirect frequency changes ultimately depend on platform-specific power management behaviour, and can produce unintuitive runtime–energy trade-offs. Rather than acting as a simple knob to trade runtime performance for energy efficiency, power limits and frequency scaling jointly reshape the internal distribution of power across components. Understanding this behaviour is essential when interpreting the effects of power limiting.

6.4. Energy efficiency is a balancing act

For I/O-bound workloads, the interaction between parallelism and power limiting differs fundamentally from the compute-bound case. Increasing parallelism increases power demand without improving runtime performance as execution is constrained by external resources, making lower thread counts generally more energy-efficient.

On the desktop system (Figure 9a), tightening the power limit initially reduces energy consumption without a large runtime cost, reflecting the limited dependency on compute performance. As the power limit tightens further, the runtime impact becomes increasingly severe, as the processor can no longer keep up with I/O throughput. Eventually, the power limit becomes so restrictive that runtime performance and energy efficiency both deteriorate. These effects occur earlier for higher thread counts, due to a higher power demand. Although the trade-off is relatively simple, it remains coupled: selecting an energy-efficient power limit depends directly on the degree of parallelism.

Moreover, despite largely monotonic behaviour on the desktop system, selecting an appropriate power limit remains non-trivial. Across thread counts, we observe extended regions in which energy efficiency remains nearly constant while runtime performance varies substantially. For example, with eight threads, power limits between 12.5 W and 37.5 W yield comparable energy efficiency, yet increasing the power limit across this range improves runtime performance by a factor of five. This behaviour shows that energy-efficient configurations are not unique. Across a range of power limits, negligible changes in energy efficiency can yield significant runtime improvements. Thus, selecting a favourable configuration requires not only joint tuning of both controls, but also joint consideration of both objectives. Z-plots make this explicit, exposing opportunities that would remain hidden when optimising for a single objective.

On the Odroid-H4 (Figure 9b) the interaction is particularly non-monotonic, producing local energy-efficiency optimums at intermediate power limits. In the sequential case, the higher-power local optimum achieves comparable energy efficiency to the lower-power local optimum, with substantially better runtime. However, as the thread count increases, the energy efficiency difference widens, making low power configurations increasingly more attractive, despite the significant runtime penalty.

Energy-efficient execution is confined to a narrow range of configurations: both overly restrictive and overly permissive power limits can degrade efficiency, and the optimal setting shifts with the degree of parallelism. In contrast to the compute-bound workload, parallelism primarily introduces excess power draw, while power limiting governs how effectively the processor sustains external resource demand. Overall, I/O-bound workloads highlight that energy efficiency depends on a careful balance between processor performance and resource demand.

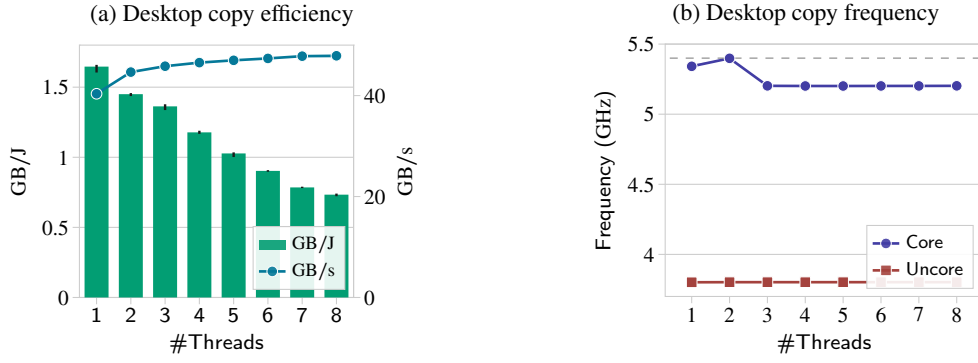


Figure 10: Desktop memory-bound workload median runtime performance and energy efficiency (a) and processor frequency (b) when adjusting the thread count, using the default power limit of 120 W. Error bars report the interquartile range.

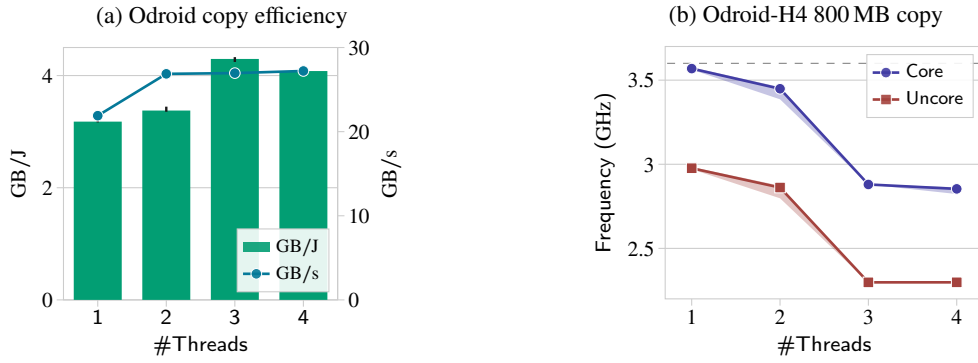


Figure 11: Odroid-H4 memory-bound workload median runtime performance and energy efficiency (a) and processor frequency (b) when adjusting the thread count, using the default power limit of 12 W. Error bars report the interquartile range.

7. Interleaved workload

Although I/O-bound and memory-bound programs appear similar at a high level, as both primarily move data, they interact with the hardware and operating system in fundamentally different ways. I/O-bound workloads are constrained by storage devices and kernel-level I/O handling, whereas memory-bound workloads are limited by on-chip memory resources. As a result, memory-bound workloads are highly sensitive to how system configuration reshapes the balance between core and uncore performance.

Thread counts influence memory throughput by increasing effective cache size and the degree of concurrency in memory requests. Power limits, in turn, determine whether the uncore can sustain the bandwidth required to serve those requests. Together, these interactions give rise to non-monotonic runtime and energy trade-offs. To study these effects we use a memory copy benchmark [31], which repeatedly reads and writes 100 million doubles from system memory.

7.1. Parallelism hides latency, at a cost

Unlike purely I/O-bound workloads, memory-bound programs can benefit from increased parallelism. Although individual threads remain limited by memory latency, increasing the number of concurrent memory requests in flight can partially hide this latency. This effect persists until

the memory subsystem becomes saturated, beyond which additional parallelism provides little benefit and may even degrade performance due to resource contention.

Figure 10a shows that slightly increased parallelism substantially improves runtime performance, after which runtime gains continue to taper off. These improvements, however, come at a significant energy cost. Activating additional cores increases power demand, while the modest decrease in frequency is not enough to offset the increased energy consumption.

Parallelism can hide memory latency and improve runtime performance, at the cost of increased power demand from activating additional cores. For this workload, using two or three threads achieves most of the attainable speedup at a comparatively small energy cost, whereas higher thread counts provide diminishing runtime benefits.

7.2. Frequency scaling offsets power demand

On the Odroid-H4, core–uncore frequency scaling fundamentally alters the relationship between parallelism and energy efficiency (Figure 11a). As thread counts increase, runtime performance continues to improve due to higher memory-level parallelism. Unlike on the desktop system, however, activating additional cores actually *improves* energy efficiency.

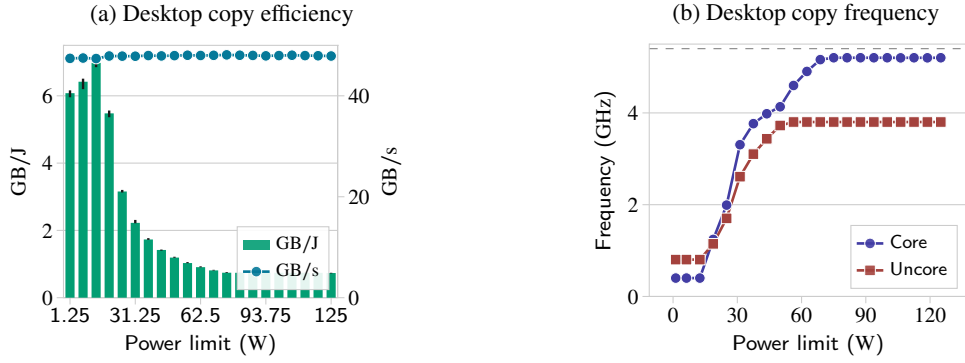


Figure 12: Desktop memory-bound workload median runtime performance and energy efficiency (a) and processor frequency (b) when adjusting the power limit, using all eight available performance cores. Error bars report the interquartile range.

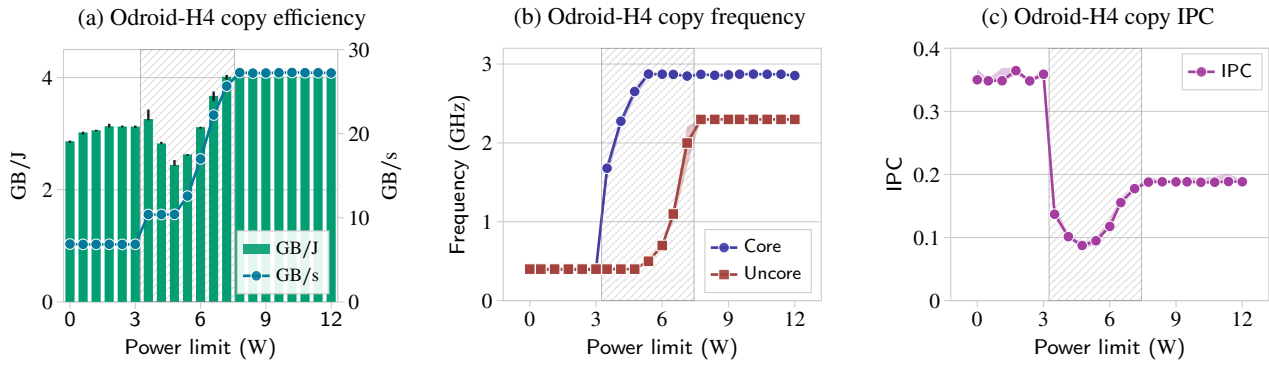


Figure 13: Odroid-H4 memory-bound workload median runtime performance and energy efficiency (a), processor frequency (b), and IPC (c) when adjusting the power limit, using all four available cores. Error bars report the interquartile range.

Figure 11b shows that an increased number of threads triggers a substantial reduction in both core and uncore frequencies. The additional power demand introduced by activating more cores is completely overcome by this aggressive downscaling. This behaviour contrasts sharply with the desktop system, where frequency remains approximately constant as thread count increases, and the power cost of activating additional cores directly translates into increased energy consumption.

This further illustrates that the same parallelism setting can lead to qualitatively different outcomes across platforms, depending on how frequency scaling and power distribution are managed at the hardware level.

7.3. Power limiting is a free lunch, for some

Tightening the power limit on the desktop system reduces both core and uncore frequencies at a similar rate (Figure 12b). Because the benchmark is light on computation but generates substantial memory traffic, runtime initially remains nearly constant while energy efficiency improves substantially due to a significant decrease in core frequency. In this configuration, lowering the power limit effectively removes excess compute capacity without affecting the memory subsystem.

On the Odroid-H4, power limiting produces more irregular behaviour (Figure 13b). As the power limit tightens, uncore frequency decreases much earlier than core frequency. At power limits between 5 W and 7 W, uncore frequency drops sharply, while the additional power headroom enables an increase in core frequency. This pattern is counterproductive, because the workload depends primarily on memory throughput and uncore performance. As a result, both runtime performance and energy efficiency deteriorate.

This shift in behaviour is also visible in the instructions per cycle (IPC) measurements shown in Figure 13c. Once the power limit falls into this 5–7 W range, IPC decreases noticeably, indicating that the processor spends more cycles waiting for data from memory as the memory subsystem becomes the dominant bottleneck.

At lower power limits, between 3 W and 5 W, uncore frequency stabilises while core frequency decreases substantially, matching uncore frequency at 3 W. Figure 13c indicates that the program becomes less memory-bound in this range. Although this slightly offsets the energy efficiency loss from the uncore frequency drop, it is not enough to present a sufficiently energy-efficient configuration.

These results highlight that energy-efficient execution of memory-bound workloads depends critically on maintaining a balance between core and uncore performance. Moderate

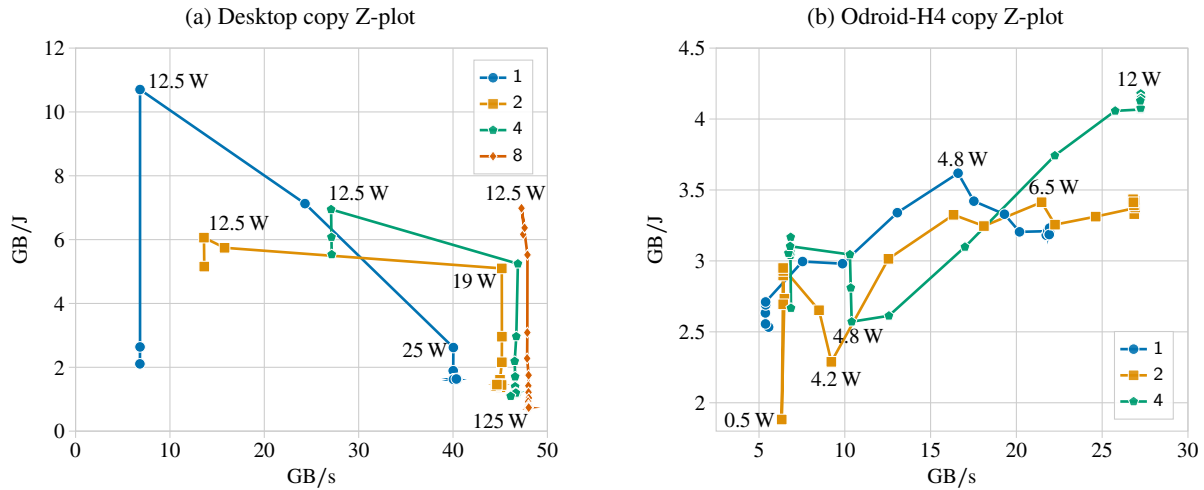


Figure 14: Z-plots combining parallelism and power limiting configurations for the memory-bound workload, on the desktop system (a) and the Odroid-H4 (b). Lines connect configurations with equal thread count, while movement along a curve from left to right reflects the effect of increasing the power limit. Exact power limits are given for relevant configurations.

power limits can yield substantial energy savings with minimal runtime impact when this balance is preserved.

7.4. Parallelism and power limiting are joint levers

The memory-bound workload most clearly exposes the joint influence of parallelism and power limiting. Here, runtime performance is limited by memory throughput, while energy consumption depends on how efficiently the processor provisions core and uncore components.

On the desktop system (Figure 14a), the Z-plot shows an interesting pattern. Across most configurations, reducing the power limit improves energy efficiency with little or no impact to runtime, indicating that execution is bounded by memory bandwidth rather than compute capacity. This results in extended vertical plateaus in the Z-plot, where runtime performance remains nearly constant while energy consumption varies substantially. The extent of these plateaus increases with parallelism, as higher thread counts compensate for reduced core frequencies by issuing more concurrent memory requests.

This behaviour is the inverse of the I/O-bound case. Whereas the I/O workload exhibits plateaus in energy efficiency with varying runtime, the memory-bound workload exhibits plateaus in runtime with varying energy consumption, reflecting fundamentally different bottlenecks.

Only at sufficiently restrictive power limits does runtime begin to degrade, once the processor can no longer saturate the available memory bandwidth. This transition depends strongly on thread count: higher parallelism increases concurrency, whereas lower thread counts require higher frequencies—and thus higher power limits—to sustain throughput due to limited latency hiding.

Despite a global energy minimum at 12.5 W across thread counts, these transitions expose practically relevant intermediate configurations. For example, in the two-thread configuration, increasing the power limit to 19 W yields a

substantial runtime improvement at a modest energy cost, providing a more balanced operating point than the absolute energy optimum.

Finally, this interaction shows that higher parallelism is not inherently more energy efficient. Although the sequential configuration generally underperforms across most power limits, careful tuning can reduce its energy consumption sufficiently to outperform parallel configurations, by matching memory demand to available bandwidth while avoiding the overhead of additional active cores.

On the Odroid-H4 (Figure 14b), the interaction becomes substantially more complex. Increasing parallelism raises memory demand through concurrent requests, while power budget determines how effectively this demand is served through core and uncore frequency scaling. The result is a tightly coupled system in which changes to either control shift the balance between computation and data movement. Consequently, configurations that are efficient at one thread count may become suboptimal at another: the Z-plot exhibits multiple local optima and frequent crossovers. Although in this case a global optimum exists, the surrounding landscape is highly irregular. This shows that efficient configurations are difficult to identify without joint tuning, mainly due to platform-specific frequency scaling characteristics.

Overall, the memory-bound workload emphasises that runtime performance and energy efficiency are governed by the balance between memory demand and memory supply. Parallelism determines the rate at which memory requests are generated, while power limiting can reduce energy consumption by downscaling underutilised components. Energy-efficient execution cannot be achieved by tuning either control in isolation: the optimal configuration crucially depends on the ability of the platform to effectively manage power distribution and performance scaling across active cores and subdomains.

8. Discussion

Across benchmarks, changes in parallelism and power limits influence runtime and energy use primarily by reshaping how hardware resources are exercised. Consequently, runtime–energy trade-offs vary substantially across workloads and platforms, indicating that energy-efficient execution cannot be understood independently of hardware power management behaviour.

Frequency scaling emerges as the dominant mechanism mediating these effects. Parallelism affects frequency indirectly through increased utilisation and shared power availability, while power limits constrain frequency directly by restricting the available power budget. However, scaling behaviour differs markedly between architectures: high-performance processors tend to downscale conservatively, whereas low-power systems apply more aggressive scaling. Consequently, identical configurations can yield qualitatively different runtime–energy trade-offs across platforms.

These effects are further shaped by how power is distributed between core and uncore domains. On platforms where these domains are managed independently, power can, in principle, be redirected toward performance-critical components. In practice, however, power management policies do not always achieve this balance. When power fails to be redirected from underutilised components toward performance-critical ones, potential energy savings remain unrealised. This effect is particularly evident for workloads that depend on multiple domains, where an imbalance in core–uncore allocation can degrade throughput without an energy efficiency increase.

Taken together, these mechanisms show that parallelism and power limits act as coupled controls. Parallelism increases pressure on shared resources and raises power demand, while power limits constrain how that demand is serviced through frequency scaling and power allocation. Consequently, tuning parallelism and power limits independently is insufficient to capture the full range of efficient configurations, as the optimal setting for one control depends on the setting of the other.

A key implication is that runtime performance and energy efficiency must be treated as joint optimisation objectives. Z-plots reveal that the configuration space is structured by extended regions in which one objective remains comparable while the other varies substantially. These regions indicate that optimal configurations are not isolated points, but rather lie within broader regions of similar performance. Within these regions, small sacrifices to one objective can yield substantial improvements in the other, revealing opportunities for more balanced operating points.

Overall, these findings show that energy optimisation is inherently a hardware-dependent and multi-objective problem. Independent tuning of parallelism and power limits risks converging to locally favourable but globally suboptimal configurations. Effective optimisation therefore requires coordinated, workload-aware tuning of both controls, guided by an understanding of runtime–energy trade-offs and the underlying hardware power management behaviour.

9. Related work

Energy-efficient execution has been studied from complementary perspectives, including measurement and attribution, optimisation through system-level control, and modelling of energy-related behaviour.

9.1. Energy measurement and attribution

Reliable energy optimisation requires accurate measurement and attribution. Prior work explores both direct measurement techniques using external power meters [32] and on-chip energy counters [25, 33], as well as model-based estimation approaches [34–36]. These approaches are often combined to balance granularity, accuracy, and practicality.

Beyond measurement, several works investigate how energy consumption can be attributed to software components. This includes code-level energy profiling [37, 38], anomaly detection [39], and model-based estimation techniques [40]. The broader organisational relevance of these contributions is further explored by studies on energy-aware software development processes [41, 42]. At a broader scale, life-cycle assessment extends energy analysis beyond execution to include manufacturing and operational costs [21, 22].

These efforts provide the foundation for energy-aware optimisation. Our work complements this by separating baseline system power from program-induced energy.

9.2. System-level optimisation

A large body of work investigates how software can dynamically adapt resource usage to improve energy efficiency. Controlling parallelism is a central theme: oversubscription can degrade both runtime performance and energy efficiency [43], while undersubscription can waste energy by leaving hardware underutilised [44–47]. This establishes parallelism as a key optimisation lever. Our results extend this view by showing how these effects are further shaped by power limits.

Dynamic adaptation techniques extend this idea by adjusting system parameters based on observed program behaviour. Examples include adaptive schedulers [48–50], utility-based resource management [1], and energy-aware runtime systems [2, 3]. At larger scales, cloud and cluster-level approaches coordinate resource allocation and power distribution across nodes [51–54]. Our results provide empirical support for these approaches, by showing that system parameters must be considered with respect to the hardware and running conditions.

Dynamic voltage and frequency scaling is a well-known mechanism for reducing energy consumption [55, 56]. Systems such as *EAR* [4–6], *Adagio* [7], and energy-aware schedulers [8–10], demonstrate that selective downscaling can reduce energy with limited performance loss.

Power limiting provides a complementary control mechanism, constraining total power rather than directly setting frequencies. Prior work shows that power limits can trade runtime for energy savings [11–14]. Our results show that the effect of power limiting depends crucially on how it interacts with frequency-scaling policies.

9.3. Modelling and multi-objective optimisation

Analytical and learning-based models have been proposed to predict energy consumption [57] and to find energy-optimal thread count and frequency configurations [58–61]. These works demonstrate that optimal configurations often arise from non-linear and platform-specific interactions.

These approaches typically formulate optimisation as a multi-objective problem, balancing runtime performance and energy consumption. Multi-objective optimisation is commonly analysed using Pareto frontiers, which characterise the set of non-dominated configurations that represent optimal trade-offs between objectives [62, 63]. In the context of high-performance computing, Pareto analysis has been used to guide energy-aware scheduling and system configuration [64–67]. Using Z-plots to visualise runtime-energy trade-offs across the configuration landscape provides a complementary perspective to Pareto analysis.

9.4. Inter-component behaviour

Previous work shows that core and uncore frequencies often scale independently [68, 69], which highlights the importance of modelling interactions between hardware domains. Our results confirm that memory-intensive workloads are particularly sensitive to the distribution of power between core and uncore domains. This motivates the need for dynamic workload classification, which can be achieved through performance counter-based techniques [70, 71].

Analytical models demonstrate that energy use depends jointly on core and uncore behaviour [72, 73], motivating joint scaling strategies. Runtime systems such as *Cuttlefish* [74] and *DUF* [75] dynamically adapt both core and uncore domains, while machine-learning-based approaches show that hardware governors do not always select energy-optimal configurations, calling for coordinated management across domains [76–78]. These effects extend to heterogeneous systems, where mismatched scaling can lead to significant energy waste [79].

Together, these studies establish that energy-efficient execution depends on understanding how workload behaviour interacts with hardware power management across multiple domains. Our work extends this line of research by empirically characterising how parallelism and power limits reshape interactions across domains.

10. Conclusion

In this paper, we investigate how parallelism and power limits jointly shape runtime–energy trade-offs across workload classes and hardware platforms. Using a measurement methodology that isolates program-induced energy from baseline power, we demonstrate that energy efficiency cannot be inferred from runtime alone. For the same workload, execution parameters such as parallelism and power limits can produce markedly different runtime–energy outcomes. Rather than assuming a fixed relationship between runtime and energy, we show that efficient configurations emerge from workload behaviour and hardware power management.

The central mechanisms behind these effects are power distribution and frequency scaling. Parallelism increases utilisation and pressure on shared resources, while power limits reshape how power is distributed across domains. Differences in these mechanisms explain why identical configurations can produce different behaviour across platforms. Consequently, energy-efficient execution depends on coordinated tuning of parallelism and power limits, rather than isolated adjustment of either control.

A central implication of our results is that efficient execution is inherently a multi-objective optimisation problem. Configurations that minimise runtime do not generally minimise energy consumption, and near-optimal runtime performance can often be achieved across a wide range of configurations with substantially different energy costs. Using Z-plots, we expose the structure of these trade-offs, revealing configurations that balance runtime and energy, rather than a single objective.

Overall, our findings demonstrate the need for joint tuning of parallelism and power limits, and motivate the use of multi-objective optimisation. Optimal configurations depend on a combination of workload characteristics and platform-specific frequency scaling and power management behaviour, which cannot be captured reliably by static policies. Therefore, static or isolated tuning strategies are insufficient. This motivates system-level approaches that coordinate parallelism and power limits dynamically, using program feedback to identify efficient operating regions and achieve portable energy efficiency across platforms.

References

- [1] D. Machovec, B. Khemka, N. Kumbhare, S. Pasricha, A.A. Maciejewski, H.J. Siegel, A. Akoglu, G.A. Koenig, S. Hariri, et al. Utility-based resource management in an oversubscribed energy-constrained heterogeneous environment executing parallel applications. *Parallel Computing*, 83:48–72, 2019. ISSN 0167-8191. doi: 10.1016/j.parco.2017.11.005.
- [2] J. Aaldering, B. van Gastel, and S.-B. Scholz. Energy-aware dynamic adaptation of runtime systems. In *Proceedings of the 26th International Symposium on Trends in Functional Programming*, TFP '25, pages 435–458. Springer, January 2025. doi: 10.1007/978-3-031-99751-8_19.
- [3] S. Gordon and S.-B. Scholz. Dynamic adaptation of functional runtime systems through external control. In *Proceedings of the 27th Symposium on the Implementation and Application of Functional Programming Languages*, IFL '15, pages 1–13, New York, NY, 2015. ACM. doi: 10.1145/2897336.2897347.
- [4] J. Corbalan, O. Vidal, L. Alonso, and J. Aneas. Explicit uncore frequency scaling for energy optimisation policies with EAR in Intel architectures. In *2021 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 572–581. IEEE, 2021. doi: 10.1109/Cluster48925.2021.00089.
- [5] J. Corbalan, L. Alonso, J. Aneas, and L. Brochard. Energy optimization and analysis with EAR. In *2020 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 464–472. IEEE, 2020. doi: 10.1109/CLUSTER49012.2020.00067.
- [6] J. Corbalan and L. Brochard. EAR: Energy management framework for supercomputers. *Barcelona Supercomputing Center (BSC) Working paper*, 2019. URL <https://www.bsc.es/sites/default/files/public/bscw2/content/software-app/technical-documentation/ear.pdf>.

- [7] B. Rountree, D.K. Lowenthal, B.R. De Supinski, M. Schulz, V.W. Freeh, and Tyler Bletsch. Adagio: Making DVS practical for complex HPC applications. In *Proceedings of the 23rd International Conference on Supercomputing*, ICS '09, pages 460–469, New York, NY, USA, 2009. ACM. doi: 10.1145/1542275.1542340.
 - [8] P. Bratek, L. Szustak, R. Wyrzykowski, and T. Olas. Reducing energy consumption using heterogeneous voltage frequency scaling of data-parallel applications for multicore systems. *Journal of Parallel and Distributed Computing*, 175:121–133, 2023. ISSN 0743-7315. doi: 10.1016/j.jpdc.2023.01.005.
 - [9] M.P. Karpowicz. Energy-efficient CPU frequency control for the Linux system. *Concurrency and Computation: Practice and Experience*, 28(2):420–437, 2016. doi: 10.1002/cpe.3476.
 - [10] L. Ponciano, A. Brito, L. Sampaio, and F. Brasileiro. Energy efficient computing through productivity-aware frequency scaling. In *Second International Conference on Cloud and Green Computing*, pages 191–198. IEEE, 2012. doi: 10.1109/CGC.2012.59.
 - [11] M. Patrou, T. Wang, W. Elwasif, M. Eisenbach, R. Miller, W. Godoy, and O. Hernandez. Power-capping metric evaluation for improving energy efficiency in HPC applications. In *International Conference on High Performance Computing*, volume 16091, pages 231–244. Springer, November 2025. doi: 10.1007/978-3-032-07612-0_18.
 - [12] S. Cunha, L. Silva, J. Saraiva, and J.P. Fernandes. Trading runtime for energy efficiency: Leveraging power caps to save energy across programming languages. In *Proceedings of the 17th International Conference on Software Language Engineering*, SLE '24, pages 130–142, New York, NY, USA, October 2024. ACM. doi: 10.1145/3687997.3695638.
 - [13] A. Marathe, P.E. Bailey, D.K. Lowenthal, B. Rountree, M. Schulz, and B.R. de Supinski. A run-time system for power-constrained HPC applications. In *International Conference on High Performance Computing*, volume 9137, pages 394–408. Springer, 2015. doi: 10.1007/978-3-319-20119-1_28.
 - [14] B. Diniz, D. Guedes, W. Meira, and R. Bianchini. Limiting the power consumption of main memory. *SIGARCH Computer Architecture News*, 35(2):290–301, June 2007. ISSN 0163-5964. doi: 10.1145/1273440.1250699.
 - [15] J. Mair, D. Eysers, Z. Huang, and H. Zhang. Myths in power estimation with performance monitoring counters. *Sustainable Computing: Informatics and Systems*, 4(2):83–93, 2014. ISSN 2210-5379. doi: 10.1016/j.suscom.2014.03.007.
 - [16] A. Jagannadharao, N. Beckage, S. Biswas, H. Egan, J. Gafur, T. Metsch, D. Nafus, G. Raffa, and C. Tripp. A beginner’s guide to power and energy measurement and estimation for computing and machine learning. *arXiv preprint*, 2024. URL <https://arxiv.org/abs/2412.17830>.
 - [17] L. Ardito, R. Coppola, M. Morisio, and M. Torchiano. Methodological guidelines for measuring energy consumption of software applications. *Scientific Programming*, 2019(1):5284645, 2019. doi: 10.1155/2019/5284645.
 - [18] T. Johann, M. Dick, S. Naumann, and E. Kern. How to measure energy-efficiency of software: Metrics and measurement results. In *2012 First International Workshop on Green and Sustainable Software (GREENS)*, pages 51–54. IEEE, 2012. doi: 10.1109/GREENS.2012.6224256.
 - [19] J.B. Guinée, S. Cucurachi, P.J.G. Henriksson, and R. Heijungs. Digesting the alphabet soup of LCA. *The International Journal of Life Cycle Assessment*, 23(7):1507–1511, 2018. doi: 10.1007/s11367-018-1478-0.
 - [20] T. Ekvall and B.P. Weidema. System boundaries and input data in consequential life cycle inventory analysis. *The International Journal of Life Cycle Assessment*, 9(3):161–171, 2004. doi: 10.1007/BF02994190.
 - [21] M.A. Curran. Life cycle assessment: A review of the methodology and its application to sustainability. *Current Opinion in Chemical Engineering*, 2(3):273–277, 2013. ISSN 2211-3398. doi: 10.1016/j.coche.2013.02.002.
 - [22] G. Finnveden, M.Z. Hauschild, T. Ekvall, J. Guinée, R. Heijungs, S. Hellweg, A. Koehler, D. Pennington, and S. Suh. Recent developments in life cycle assessment. *Journal of Environmental Management*, 91(1):1–21, 2009. ISSN 0301-4797. doi: 10.1016/j.jenvman.2009.06.018.
 - [23] J.M. Kim, Y.G. Kim, and S.W. Chung. Stabilizing CPU frequency and voltage for temperature-aware DVFS in mobile devices. *IEEE Transactions on Computers*, 64(1):286–292, 2015. doi: 10.1109/TC.2013.188.
 - [24] Anonymous author(s). Authors and doi will be added upon acceptance. Artefact containing code sources and raw results. May 2026. URL https://zenodo.org/records/20286989?preview=1&token=eyJhbGciOiJIUzUxMiIsImIhdCI6MTc3OTEyNTg3MjZlbnRvbmVJoxNzk4NzYxNTk5fQ.eyJpZCI6ImJiYWNmWjdILThNmItNDlmZi04NWJiLWwvbjNiZWZlcnRldGE1dGVzcyJyYW50eGwidiOY0jcF6gmDStJRdBz5dlirtvAgc3YrYkgSAJzELZvi02ze5vjS904_XXnq2SKid_k7V_nJ8XMx5k5qw.
 - [25] K.N. Khan, M. Hirki, T. Niemi, J.K. Nurminen, and Z. Ou. RAPL in action: Experiences in using RAPL for power measurements. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, 3(2):1–26, June 2018. ISSN 2376-3639. doi: 10.1145/3177754.
 - [26] Texas Instruments. *INA260 Precision Digital Current and Power Monitor with Low-drift, Precision Integrated Shunt Data-sheet (Rev. C)*. Texas Instruments, Dallas, TX, USA, December 2016. Available at <https://www.ti.com/lit/gpn/ina260>.
 - [27] C. Grellck. Single assignment C (SaC) high productivity meets high performance. In *Central European Functional Programming School, CEF ’11*, pages 207–278, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg. doi: 10.1007/978-3-642-32096-5_5.
 - [28] C. Grellck and S.-B. Scholz. SaC-A functional array language for efficient multi-threaded execution. *International Journal of Parallel Programming*, 34(4):383–427, August 2006. ISSN 1573-7640. doi: 10.1007/s10766-006-0018-x.
 - [29] S.-B. Scholz. Single assignment C: Efficient support for high-level array operations in a functional setting. *Journal of Functional Programming*, 13(6):1005–1059, 2003. doi: 10.1017/S0956796802004458.
 - [30] A. Afzal. The cost of computation: Metrics and models for modern multicore-based systems in scientific computing. Master’s thesis, Department Informatik, University Erlangen-Nürnberg, July 2015.
 - [31] J.D. McCalpin. Memory bandwidth and machine balance in current high-performance computers. *IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter*, 2:19–25, December 1995.
 - [32] D. Hackenberg, T. Ilsche, R. Schöne, D. Molka, M. Schmidt, and W.E. Nagel. Power measurement techniques on standard compute nodes: A quantitative comparison. In *International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 194–204. IEEE, 2013. doi: 10.1109/ISPASS.2013.6557170.
 - [33] A. Bakker. Comparing energy profilers for Android. In *21st Twente Student Conference on IT*, volume 21, Twente, Netherlands, June 2014. University of Twente. URL <https://fmt.ewi.utwente.nl/media/135.pdf>.
 - [34] L.M. Duarte, D. da Silva Alves, B.R. Toresan, P.H. Maia, and D. Silva. A model-based framework for the analysis of software energy consumption. In *Proceedings of the 33rd Brazilian Symposium on Software Engineering*, SBES ’19, pages 67–72, New York, NY, USA, 2019. ACM. doi: 10.1145/3350768.3353813.
 - [35] K. Eder, J.P. Gallagher, G. Fagas, L. Gammaitoni, and D. Paul. *Energy-Aware Software Engineering*. IntechOpen, Rijeka, Croatia, 2017.
 - [36] C. Stier, A. Kozirolek, H. Groenda, and R. Reussner. Model-based energy efficiency analysis of software architectures. In *European Conference on Software Architecture*, pages 221–238. Springer, 2015. doi: 10.1007/978-3-319-23727-5_18.
 - [37] R. Verdecchia, A. Guldner, Y. Becker, and E. Kern. Code-level energy hotspot localization via naive spectrum based testing. In *Advances and New Trends in Environmental Informatics*, pages 111–130, New York, NY, USA, 2018. Springer. doi: 10.1007/978-3-319-99654-7_8.

- [38] D. Li, S. Hao, W.G.J. Halfond, and R. Govindan. Calculating source line level energy information for Android applications. In *Proceedings of the 2013 International Symposium on Software Testing and Analysis*, ISSTA '13, pages 78–89, New York, NY, USA, 2013. ACM. doi: 10.1145/2483760.2483780.
- [39] M. Couto, T. Carção, J. Cunha, J.P. Fernandes, and J. Saraiva. Detecting anomalous energy consumption in Android applications. In *Proceedings of the 18th Brazilian Symposium on Programming Languages*, volume 18 of *SBLP '14*, pages 77–91, New York, NY, USA, October 2014. Springer. doi: 10.1007/978-3-319-11863-5_6.
- [40] M. Klinik, B. van Gastel, C. Kop, and M. van Eekelen. Skylines for symbolic energy consumption analysis. In *Formal Methods for Industrial Critical Systems*, FMICS '20, pages 93–112, New York, NY, USA, September 2020. Springer. doi: 10.1007/978-3-030-58298-2_3.
- [41] G. Pinto and F. Castor. Energy efficiency: A new concern for application software developers. *Communications of the ACM*, 60(12): 68–75, December 2017. ISSN 0001-0782. doi: 10.1145/3154384.
- [42] C. Pang, A. Hindle, B. Adams, and A.E. Hassan. What do programmers know about software energy consumption? *IEEE Software*, 33(3):83–89, 2016. doi: 10.1109/MS.2015.83.
- [43] C. Iancu, S. Hofmeyr, F. Blagojević, and Y. Zheng. Oversubscription on multicore processors. In *IEEE International Symposium on Parallel & Distributed Processing (IPDPS)*, pages 1–11. IEEE, 2010. doi: 10.1109/IPDPS.2010.5470434.
- [44] S. Maloney, E. Suarez, N. Eicker, F. Guimarães, and W. Frings. Analyzing HPC monitoring data with a view towards efficient resource utilization. In *36th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, pages 170–181. IEEE, 2024. doi: 10.1109/SBAC-PAD63648.2024.00023.
- [45] H. Huang, J. Rao, S. Wu, H. Jin, H. Jiang, H. Che, and X. Wu. Towards exploiting CPU elasticity via efficient thread oversubscription. In *Proceedings of the 30th International Symposium on High-Performance Parallel and Distributed Computing*, HPDC '21, pages 215–226, New York, NY, USA, 2021. ACM. doi: 10.1145/3431379.3460641.
- [46] W. Heirman, T.E. Carlson, K. van Craeynest, I. Hur, A. Jaleel, and L. Eeckhout. Undersubscribed threading on clustered cache architectures. In *20th International Symposium on High Performance Computer Architecture (HPCA)*, pages 678–689. IEEE, 2014. doi: 10.1109/HPCA.2014.6835975.
- [47] G. Chadha, S. Mahlke, and S. Narayanasamy. When less is more (LIMO): Controlled parallelism for improved efficiency. In *Proceedings of the 2012 International Conference on Compilers, Architectures and Synthesis for Embedded Systems*, CASES '12, pages 141–150, New York, NY, USA, 2012. ACM. doi: 10.1145/2380403.2380431.
- [48] T. Harris and S. Kaestle. Callisto-RTS: Fine-grain parallel loops. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*, pages 45–56, Santa Clara, CA, July 2015. USENIX Association. URL <https://www.usenix.org/conference/atc15/technical-session/presentation/harris>.
- [49] V.R. Segovia. *Adaptive CPU Resource Management for Multicore Platforms*. PhD thesis, Department of Automatic Control, Lund Institute of Technology, Lund University, 2011. URL <https://lup.lub.lu.se/record/2155045>.
- [50] H. Pan, B. Hindman, and K. Asanović. Composing parallel software efficiently with Lithe. In *Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '10, pages 376–387, New York, NY, USA, 2010. ACM. doi: 10.1145/1806596.1806639.
- [51] B. Reidys, P. Zardoshti, Í. Goiri, C. Irvine, D.S. Berger, H. Ma, K. Arya, E. Cortez, T. Stark, et al. Coach: Exploiting temporal patterns for all-resource oversubscription in cloud platforms. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, volume 1 of *ASPLOS '25*, pages 164–181, New York, NY, USA, 2025. ACM. doi: 10.1145/3669940.3707226.
- [52] J. Krzywda, A. Ali-Eldin, T.E. Carlson, P.-O. Östberg, and E. Elmröth. Power-performance tradeoffs in data center servers: DVFS, CPU pinning, horizontal, and vertical scaling. *Future Generation Computer Systems*, 81:114–128, April 2018. ISSN 0167-739X. doi: 10.1016/j.future.2017.10.044.
- [53] J. von Kistowski, J. Beckett, K.-D. Lange, H. Block, J.A. Arnold, and S. Kounev. Energy efficiency of hierarchical server load distribution strategies. In *2015 IEEE 23rd International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems*, pages 75–84, Piscataway, NJ, 2015. IEEE. doi: 10.1109/MASCOTS.2015.11.
- [54] D. Williams, H. Jamjoom, Y.-H. Liu, and H. Weatherspoon. Overdriver: Handling memory overload in an oversubscribed cloud. *SIGPLAN Notices*, 46(7):205–216, March 2011. ISSN 0362-1340. doi: 10.1145/2007477.1952709.
- [55] J. Balladini, R. Suppi, D. Rexachs, and E. Luque. Impact of parallel programming models and CPUs clock frequency on energy consumption of HPC systems. In *9th IEEE/ACS International Conference on Computer Systems and Applications (AICCSA)*, pages 16–21. IEEE, 2011. doi: 10.1109/AICCSA.2011.6126618.
- [56] E. Le Sueur and G. Heiser. Dynamic voltage and frequency scaling: The laws of diminishing returns. In *Proceedings of the 2010 International Conference on Power Aware Computing and Systems*, pages 1–8, October 2010.
- [57] B. Goel and S.A. McKee. A methodology for modeling dynamic and static power consumption for multicore processors. In *International Symposium on Parallel and Distributed Processing (IPDPS)*, pages 273–282. IEEE, May 2016. doi: 10.1109/IPDPS.2016.118.
- [58] A. Cabrera, F. Almeida, D. Castellanos-Nieves, A. Oleksiak, and V. Blanco. Energy efficient power cap configurations through Pareto front analysis and machine learning categorization. *Cluster Computing*, 27(3):3433–3449, 2024. doi: 10.1007/s10586-023-04151-2.
- [59] J. Chen, M. Manivannan, M. Abduljabbar, and M. Pericas. ERASE: Energy efficient task mapping and resource management for work stealing runtimes. *ACM Transactions on Architecture and Code Optimization (TACO)*, 19(2):1–29, March 2022. ISSN 1544-3566. doi: 10.1145/3510422.
- [60] V.R.G. Silva, A.F.A. Furtunato, L. Georgiou, C.A.V. Sakuyama, K. Eder, and S. Xavier-de Souza. Energy-optimal configurations for single-node HPC applications. In *2019 International Conference on High Performance Computing & Simulation (HPCS)*, pages 448–454. IEEE, 2019. doi: 10.1109/HPCS48598.2019.9188110.
- [61] V. Sundriyal and M. Sosonkina. Modeling of the CPU frequency to minimize energy consumption in parallel applications. *Sustainable Computing: Informatics and Systems*, 17:1–8, 2018. ISSN 2210-5379. doi: 10.1016/j.suscom.2017.12.002.
- [62] C.A. Coello, G.B. Lamont, and D.A. van Veldhuizen. *Evolutionary Algorithms for Solving Multi-Objective Problems: Second Edition*. Genetic and Evolutionary Computation. Springer, New York, NY, USA, 2 edition, August 2007. ISBN 978-0-387-36797-2. doi: 10.1007/978-0-387-36797-2.
- [63] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. A fast and elitist multi-objective genetic algorithm: NSGA-II. *Transactions on Evolutionary Computation*, 6(2):182–197, 2002. doi: 10.1109/4235.996017.
- [64] K.C. Adimora and H. Sun. HARMONIC: Uncertainty-aware multi-objective optimization for energy-efficient HPC resource management. *Transactions on Parallel and Distributed Systems*, 36(11): 2438–2450, 2025. doi: 10.1109/TPDS.2025.3610354.
- [65] J. Chen, M. Manivannan, B. Goel, and M. Pericàs. JOSS: Joint exploration of cpu-memory dvfs and task scheduling for energy efficiency. In *Proceedings of the 52nd International Conference on Parallel Processing*, ICPP '23, pages 828–838, New York, NY, USA, 2023. ACM. doi: 10.1145/3605573.3605586.
- [66] B. Kocot, P. Czarnul, and J. Proficz. Energy-aware scheduling for high-performance computing systems: A survey. *Energies*, 16(2):890, 2023. ISSN 1996-1073. doi: 10.3390/en16020890.

- [67] M. Guzek, J.E. Pecero, B. Dorronsoro, and P. Bouvry. Multi-objective evolutionary algorithms for energy-aware scheduling on distributed computing systems. *Applied Soft Computing*, 24:432–446, 2014. ISSN 1568-4946. doi: 10.1016/j.asoc.2014.07.010.
- [68] Z. Zheng, S. Sultanov, M.E. Papka, and Z. Lan. Exploring uncore frequency scaling for heterogeneous computing. *arXiv preprint*, 2025. URL <https://arxiv.org/abs/2502.03796>.
- [69] V. Sundriyal, M. Sosonkina, B.M. Westheimer, and M. Gordon. Uncore frequency scaling vs dynamic voltage and frequency scaling: A quantitative comparison. *Society for Modeling & Simulation International (SCS) SpringSim-HPC*, April 2018.
- [70] S. Abbasi, A. Wong, and M.J. Shafiee. MAPLE: Microprocessor a priori for latency estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2747–2756. IEEE, 2022. doi: 10.48550/arXiv.2111.15106.
- [71] D. Molka, R. Schöne, D. Hackenberg, and W.E. Nagel. Detecting memory-boundedness with hardware performance counters. In *Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering, ICPE '17*, pages 27–38, New York, NY, USA, 2017. ACM. doi: 10.1145/3030207.3030223.
- [72] C.L. Alappat, J. Hofmann, G. Hager, H. Fehske, A.R. Bishop, and G. Wellein. Understanding HPC benchmark performance on Intel Broadwell and Cascade Lake processors. In *International Conference on High Performance Computing*, pages 412–433. Springer, 2020. doi: 10.1007/978-3-030-50743-5_21.
- [73] J. Hofmann, G. Hager, and D. Fey. On the accuracy and usefulness of analytic energy models for contemporary multicore processors. In *International Conference on High Performance Computing*, pages 22–43. Springer, 2018. doi: 10.1007/978-3-319-92040-5_2.
- [74] S. Kumar, A. Gupta, V. Kumar, and S. Bhalachandra. Cuttlefish: Library for achieving energy efficiency in multicore parallel programs. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '21*, pages 1–14, New York, NY, USA, 2021. ACM. doi: 10.1145/3458817.3476163.
- [75] É. André, R. Dulong, A. Guermouche, and F. Trahay. DUF: Dynamic uncore frequency scaling to reduce power consumption. *Concurrency and Computation: Practice and Experience*, 34(3):e6580, 2022. doi: 10.1002/cpe.6580.
- [76] B. Xiao, J. Yang, and X. Qi. Imitation learning-based performance–power trade-off uncore frequency scaling policy for multicore system. *Sensors*, 23(3), 2023. ISSN 1424-8220.
- [77] Y. Wang, W. Zhang, M. Hao, and Z. Wang. Online power management for multi-cores: A reinforcement learning based approach. *IEEE Transactions on Parallel and Distributed Systems*, 33(4):751–764, 2021. doi: 10.1109/TPDS.2021.3092270.
- [78] D.-C. Juan and D. Marculescu. Power-aware performance increase via core/uncore reinforcement control for chip-multiprocessors. In *Proceedings of the 2012 ACM/IEEE International Symposium on Low Power Electronics and Design, ISLPED '12*, pages 97–102, New York, NY, USA, 2012. ACM. doi: 10.1145/2333660.2333686.
- [79] Z. Zheng, S. Sultanov, M.E. Papka, and Z. Lan. Minimizing power waste in heterogeneous computing via adaptive uncore scaling. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '25*, pages 505–518, New York, NY, USA, 2025. ACM. doi: 10.1145/3712285.3759879.